VIPL-HR Integration — I/O Adapter Guideline

Spandan: Contactless Vital Sign Monitoring — EEE 312 DSP Project, BUET

Team Spandan

August 8, 2026

Contents

What This Note Is (and Isn't)

This is an **I/O integration note**, not new DSP theory — Segments 2-5 (face/ROI extraction, filtering, CHROM/POS/FFT heart rate, SpO2 ratio-of-ratios + calibration) are untouched. Everything here is about getting a second dataset, VIPL-HR, feeding into that same, already-tested pipeline through two new loader functions. If you already understand Segments 2-5, this document adds nothing new about the DSP itself.

Why VIPL-HR, and What It Adds

DATASET_1 (UBFC) gave this project SpO2 ground truth for only **5 subjects**, spanning a narrow 96-99% true range — thin enough that the Segment 5 Guideline PDF calls it a "calibration attempt," not a validated calibration. VIPL-HR ships **107 subjects**, each with real HR and SpO2 ground truth recorded per second by a CONTEC CMS60C pulse oximeter (the same sensor family already used for DATASET_1), across up to 9 recording scenarios per subject (stable, motion, talking, dark, bright, long-distance, post-exercise, and two phone-camera scenarios). Even a modest first slice of VIPL-HR gives Segment 6's leave-one-subject-out validation dramatically more subjects and more physiological variety to work with than DATASET_1 alone ever could.

No Algorithm Code Changed — Why That Was Possible

`extractROISignals.m`, `bandpassClean.m`, `chromCombine.m`, `posCombine.m`, `fftHeartRate.m`, `ratioOfRatios.m`, and `calibrateSpO2.m` all already take plain numeric signals and a sampling rate — nothing UBFC-specific is baked into any of them. The only place a new dataset's real format has to be reconciled is at the **input boundary**: turning VIPL-HR's actual files into the same `(R, G, B, fs)` / ground-truth shapes those functions already expect. That is exactly what `io/loadVIPLVideo.m` and `io/loadVIPLGroundTruth.m` do, and it is the only place this integration needed to write new code.

Common Beginner Mistake

Common beginner mistake. Assuming a new dataset "not fitting" the existing functions means the functions need new dataset-aware branches. The correct fix is almost always on the loader side: reshape the new data to match what the existing, tested functions already expect. Adding a `if strcmp(dataset, 'VIPL')` branch inside `chromCombine.m` would make that function harder to test and reason about for *every* future dataset, not just this one.

The One Real Surprise: VIPL-HR's Video Frame Rate Cannot Be Trusted Naively

This project's standing rule, learned from UBFC, is "always read fps from `VideoReader.FrameRate`, never hardcode it." Applied to real VIPL-HR files, that rule turned out to be **necessary but not sufficient**.

Every VIPL-HR video tested reported `VideoReader.FrameRate = 25`, regardless of which of the four recording devices made it — including the RealSense color/NIR cameras, whose own dataset documentation claims "~30 fps." Cross- checking against each video's own frame-acquisition-timestamp file (`time.txt`, shipped alongside 3 of the 4 sources) showed the declared 25 fps was measurably wrong for some recordings — by as much as **19-20%** — while matching almost exactly for others. VIPL-HR's own documentation explains why: these videos are re-encoded for storage, and the container's declared playback rate is a compression artifact, not real capture timing; real timing lives in `time.txt`.

Intuition First

Intuition first. Think of `time.txt` as the camera's own logbook of exactly when each frame was actually taken, and the video container's declared frame rate as just the number a video player uses to decide how fast to play the file back smoothly. Those two numbers usually agree, but nothing forces them to — a video can be re-packaged to play back at a tidy round number (25 fps) while the frames inside it were genuinely captured irregularly or at a different real rate. `heartrate/fftHeartRate.m` needs the *real* capture rate to convert a frequency correctly to bpm; the playback rate is the wrong number to feed it whenever the two disagree.

`loadVIPLVideo.m` handles this by recomputing frame rate from `time.txt` whenever one exists, falling back to `VideoReader.FrameRate` only for the one source (the phone camera) that ships no `time.txt` at all — a real, documented accuracy limitation for that source specifically, not an oversight. See `docs/VIPL_DATA_FORMAT.md` Section 4 for the full measured numbers, and `VIPL_Integration_LineByLine_Explanation.md` Part 1 for the code itself.

The VIPL_ Subject ID Convention, and Why It Matters for Segment 6

VIPL-HR has no single "the video" per subject the way UBFC does — every `(subject, scenario, source)` triple is its own independent recording with its own ground truth. This integration's subject IDs are built as `VIPL_p<N>_v<scenario>_source<S>` (e.g. `VIPL_p1_v1_source1`), saved into the same `data/processed/` folder UBFC subjects already use.

This matters for two reasons. First, **collision avoidance**: UBFC subject IDs look like `5-gt`, `subject5`, `after-exercise` — none of those could ever collide with a `VIPL_`-prefixed ID, so both datasets' processed `.mat` files can sit in the same shared folder with zero risk of one silently overwriting the other. Second, **dataset provenance stays visible downstream**: Segment 6 can tell which dataset any given row came from just by looking at the subject ID string, without needing a side lookup table — and the HR/SpO2 summary CSVs this integration writes (`segment4_hr_summary_vipl.csv`, `segment5_vipl_calibration.csv`) carry an explicit `dataset` column on top of that, for the same reason.

Small-Scale Validation Result (3 Subjects, Before Any Larger Batch Run)

Per this project's standing discipline, the new loaders were validated end-to-end on 3 real VIPL-HR subjects (`p1`, `p2`, `p3`, all scenario `v1` "stable", source1 webcam) before any larger batch was attempted — including manually confirming `vision.CascadeObjectDetector()` still finds a face on VIPL-HR's frames, which arrive **already cropped tight to the face** by the dataset's own authors (a structurally different starting point from UBFC's full-scene video). The detector worked without modification on all 3 subjects (one subject had 4 out of 754 frames briefly fall back to a reused bounding box — the same graceful-degradation path `extractROISignals.m` already has for UBFC, not a new failure mode).

All 3 rows are scenario `v1` (stable), source1 (webcam) — full subject IDs are `VIPL_p1_v1_source1`, `VIPL_p2_v1_source1`, `VIPL_p3_v1_source1`.

Subject	HR (CHROM)	HR ground truth	SpO2 (predicted, LOO)	SpO2 true
p1	62.1 bpm	63.1 bpm	98.1%	96.2%
p2	76.1 bpm	82.0 bpm	102.1%	96.0%
p3	69.4 bpm	74.5 bpm	96.1%	97.6%

CHROM/POS heart rate estimates landed within about 1-6 bpm of ground truth across all 3 subjects, and consistently beat green-channel-only estimation — the same pattern already established on UBFC in Segment 4. The 3-subject leave-one-out SpO2 calibration produced a 3.1 percentage-point mean absolute error, in the same rough ballpark as DATASET_1's own 5-subject result (1.97 points) — and comes with exactly the same "thin data" caveat the Segment 5 Guideline PDF already documents for DATASET_1: 3 subjects proves the *code path* works end-to-end on real VIPL-HR data, it does not constitute a validated calibration. That is Segment 6's job, across the full available subject pool.

What's Next: Segment 6

This integration's whole purpose was to unblock Segment 6 — the project's final, formal leave-one-subject-out validation (`validation/runLOSO.m`/`computeMetrics.m`/`blandAltman.m`, still untouched stubs) — with a genuinely larger, more varied pool than DATASET_1's 5 subjects could ever provide on its own. See `vipl_integration/VIPL_Team_README.md` for how to extract your own share of VIPL-HR's 107 subjects and run `scripts/run_vipl_integration_batch.m` against them.